

Feature Engineering Documentation

Ezitech Engineering Framework — Case Study AI-005

AI-Powered Internship Performance Prediction & Risk Analytics Platform

Prepared by: Sadaf Riaz & Saira Ejaz

1. Purpose and Scope

This document describes, in full, the feature engineering work carried out for the internship performance prediction system: the raw data it starts from, every engineered feature computed from that data, the reasoning behind each design decision, and exactly which features feed into each of the platform's four machine learning models. It is scoped strictly to what was implemented in `app/ml/features.py` and the four model-specific `build_*.py` files — no feature or design choice described here is hypothetical or planned; everything below reflects the code as it exists in the repository.

Feature engineering is the layer that sits between raw database records and every model in this system. Six activity tables — attendance, tasks, github_activity, code_reviews, mentor_feedback, and communication_activity — are recorded at a fine-grained, per-event level (one row per day, per task, per commit, per review). None of that raw form is directly usable by a model. The purpose of this pipeline is to convert that raw activity, for each intern, into a single row of meaningful numeric signals that a classifier or regressor can actually learn from.

2. Design Principle: One Shared Pipeline

All full-history feature computation is centralized in one module, `app/ml/features.py`, rather than being reimplemented separately inside each model's training script. Two functions form the shared interface used across the system:

- `build_feature_vector(intern_id, db)` — computes the full feature set for a single intern. This is the function used at prediction time, when the API needs a live feature row for one specific intern.
- `build_features_dataframe(db, min_days)` — calls `build_feature_vector()` for every eligible intern and returns one combined table, used when building a training dataset.

Keeping this logic in one place matters for a concrete reason: if feature computation were duplicated across training code and prediction code, the two copies could quietly drift apart over time, so that a live prediction would be built from slightly different numbers than the ones a model was actually trained on. Reusing one function for both eliminates that entire category of bug by construction.

3. Raw Data Sources

Every engineered feature below is derived from one or more of these six tables:

Table	Granularity	Used For
attendance	one row per intern per day	attendance_rate, attendance_trend
tasks	one row per assigned task	task_completion_rate, late/skipped ratios, hard_task_completion_rate, completion_trend
github_activity	one row per intern per day	commit/PR/issue totals, avg_commits_per_active_day, commit_std
code_reviews	one row per completed/late task	avg_code_review_score, review_score_std, review_trend
mentor_feedback	one row per feedback entry	avg_mentor_rating
communication_activity	one row per intern per day	avg_messages_sent, meeting_attendance_rate

4. Engineered Features

The table below lists every feature produced by `_aggregate()` inside `features.py`, computed once per intern from that intern's complete history on file.

Feature	Computed As
attendance_rate	present days ÷ total attendance days on record
task_completion_rate	completed tasks ÷ total tasks assigned
late_task_ratio	late tasks ÷ total tasks assigned
skipped_task_ratio	skipped tasks ÷ total tasks assigned
total_commits	sum of GitHub commits across all recorded days
total_pull_requests	sum of GitHub pull requests
total_issues_opened	sum of GitHub issues opened
total_issues_closed	sum of GitHub issues closed

avg_commits_per_active_day	total commits ÷ number of days with any GitHub activity
avg_code_review_score	mean of all code review scores received
review_score_std	standard deviation of code review scores
commit_std	standard deviation of daily commit counts
hard_task_completion_rate	completed hard-difficulty tasks ÷ total hard-difficulty tasks
attendance_completion_interaction	attendance_rate multiplied by task_completion_rate
on_time_given_turned_in	completed tasks ÷ (completed tasks + late tasks)
avg_mentor_rating	mean of all mentor feedback ratings
avg_messages_sent	mean messages sent per communication record
meeting_attendance_rate	mean of the meetings-attended flag
attendance_trend	average attendance in the second half of the intern's history minus the first half
completion_trend	average task completion in the second half minus the first half
review_trend	average review score in the second half minus the first half
days_since_start	reference date minus the intern's internship start date
internship_progress_pct	days_since_start divided by 90, capped at 1.0

5. Why These Specific Features Were Chosen

5.1 Rate-based features over raw counts

attendance_rate and task_completion_rate are expressed as rates rather than raw totals deliberately: an intern who joined 10 days ago and one who joined 80 days ago will have very different raw attendance counts even if both attend equally reliably. Rates make interns directly comparable regardless of how long they have been in the program.

5.2 Consistency features (review_score_std, commit_std)

An average alone can hide instability. Two interns can share the same avg_code_review_score while one delivers steadily and the other swings between very high and very low scores. review_score_std and commit_std were added specifically to expose that difference, since consistency is itself a meaningful signal of risk or strength that a plain mean discards.

5.3 Difficulty-weighted performance (hard_task_completion_rate)

Raw task_completion_rate treats an easy task and a hard task as equally meaningful, which can flatter an intern who completes many easy tasks while avoiding hard ones. hard_task_completion_rate isolates performance specifically on hard-difficulty tasks, giving a more honest read of technical capability.

5.4 Interaction and conditional features

attendance_completion_interaction (attendance_rate × task_completion_rate) captures a non-linear pattern: an intern who is both frequently absent and behind on tasks is a stronger risk signal than either issue alone. on_time_given_turned_in narrows this further, separating interns who are slow-but-reliable from those who are fast but skip work, since both would otherwise look similar under task_completion_rate alone.

5.5 Trend features (attendance_trend, completion_trend, review_trend)

These were added specifically because the underlying data generator ties dropout likelihood to whether an intern's overall trajectory is declining or improving, not only to their average level of performance. A whole-history average by itself cannot distinguish a consistently mediocre intern from one who started strong and is now declining sharply — the two can share an identical average despite representing very different real situations. Comparing the second half of an intern's history against the first half exposes that directional information directly.

5.6 Time-position features (days_since_start, internship_progress_pct)

These give every model a sense of where an intern currently stands within their internship timeline, since a low task count is expected and normal for an intern in their second week but would be a genuine warning sign in week ten. Without this context, the same raw numbers could be read as either normal or concerning depending on timing that the model would otherwise have no way to know.

6. Missing Value Handling

Two layers of missing-value handling are applied. First, features.py itself avoids most missing values structurally: rates and averages use guarded division (falling back to a default of 0 or 1 in the denominator) so that an intern with, for example, zero recorded GitHub activity produces a defined value of 0 rather than an undefined or crashing computation. Second, every model-specific build_*.py file

applies an explicit `fillna(0)` as a final safeguard immediately before the data reaches a model, so that any column left unset for a given intern is treated as "no recorded activity" rather than silently dropped or left null.

7. Categorical Encoding

The technology column — Laravel, MERN Stack, Machine Learning, Flutter, UI/UX, or DevOps — is the only categorical feature in the pipeline. It is one-hot encoded via `pandas.get_dummies` into six binary `tech_*` columns (for example `tech_Laravel`, `tech_Flutter`), letting tree-based models split on technology track directly rather than needing it as a single numeric-coded column.

8. Reference Date Handling

Several features (`days_since_start`, `internship_progress_pct`) and the training eligibility cutoff depend on a notion of "today." Using the real-world calendar date for this was tried first and found to be a defect: the same, unchanged database, queried on two different days, would silently produce two different sets of results with no error raised. This was corrected by deriving the reference date from the data itself — the most recent attendance date on file — so results only change when the underlying data changes.

9. Minimum History Requirement

An intern must have at least 14 days of recorded history (`MIN_DAYS_FOR_TRAINING`) before being included in any training dataset. Interns below this threshold are excluded, since with only a few days on record, computed rates and trend features would be based on so little data that they would not meaningfully reflect the intern's actual behaviour.

10. Per-Model Feature Sets

Every model draws from the shared feature table above, but each one removes a different subset of columns before training, specifically to prevent that model from being able to see — directly or indirectly — the same information used to construct its own label. This section documents exactly what each model uses and excludes.

10.1 Model 1 — Dropout Risk

Uses the full shared feature set. Only `status` and `completed` are removed, since `status` is where the dropped label itself comes from, and `completed` is the complementary outcome that would otherwise let the model simply read off the answer for any intern who did not drop out. Only interns with a known final outcome (`status = dropped` or `completed`) are used for training; interns still active are excluded, since their eventual outcome is not yet known and including them would introduce incorrect labels.

10.2 Model 2 — Success Probability

Built from its own SQL aggregation rather than the shared `features.py` pipeline, computing: `attendance_rate`, `task_completion_rate`, `late_tasks`, `skipped_tasks`, `total_commits`, `total_prs`, `avg_review_score`, `num_reviews`, `avg_mentor_rating`, `total_messages`, `total_meetings`, and one-hot technology columns.

The label is defined as `success = 1` when `status` is `completed` AND `avg_review_score` is at or above the median score among completed interns, and 0 otherwise — this defines success as genuinely strong completion, not merely finishing. `avg_review_score` is removed from the feature set after the label is built, since it was the value used to construct that label directly and would otherwise leak the answer.

10.3 Model 3 — Performance Trend

Uses the shared feature set with the following columns removed:

- `attendance_rate` and `task_completion_rate` — these are used to build the label itself
- `late_task_ratio` and `skipped_task_ratio` — these two always sum with `task_completion_rate` to exactly 1, so leaving them in would let the model reconstruct `task_completion_rate` almost exactly through simple arithmetic
- `attendance_completion_interaction` — this column is literally `attendance_rate` multiplied by `task_completion_rate`
- `on_time_given_turned_in` — derived from the same `completed/late` task counts the label is built from

The label itself is a composite score, computed as $0.5 \times \text{attendance_rate} + 0.5 \times \text{task_completion_rate}$, then ranked and split into three equally-sized tiers: declining, stable, and improving.

10.4 Models 4 and 5 — Learning Speed and Skill Growth

Built with a dedicated early/late split pipeline, separate from `features.py`. For each intern, attendance, task completion, GitHub commits, review scores, and mentor ratings are each divided into an early half and a late half of that intern's history.

The feature set consists of: `attendance_early`, `completion_early`, `commits_early`, `review_early`, `mentor_rating_early`, `attendance_early_trend`, and `completion_early_trend` — the last two computed by splitting the early half itself into two quarters, revealing whether a trend is already visible within the early period alone.

Two continuous labels are produced: `learning_speed = completion_late` minus `completion_early`, and `skill_growth = review_late` minus `review_early`. Only interns with at least 6 attendance records and 2 task records are included, since a meaningful split needs enough data on both sides.